Phish360 · Sprint 2
Deep learning, fusion et comparaison des modèles

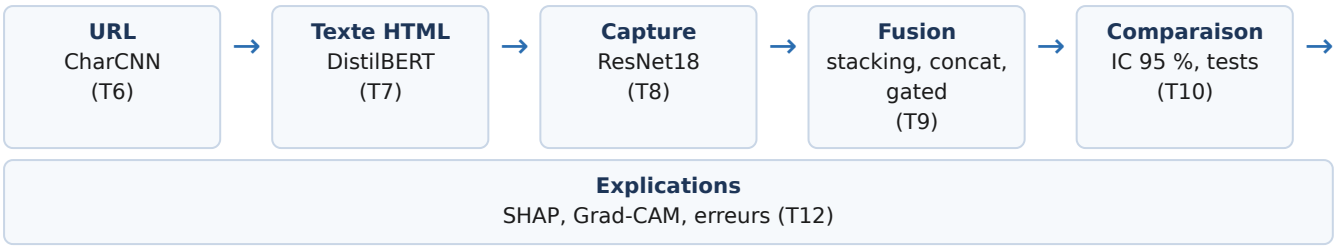
Guide pas à pas : installations, commandes, sorties attendues, recommandations

Tâches 6 à 12 · Windows + VS Code + GPU NVIDIA (RTX 3050 6 Go) · novembre 2026

1. Vue d'ensemble du Sprint 2
2. Ajouter les fichiers au projet
3. Installations (PyTorch CUDA, transformers, SHAP…)
4. Test à blanc avec des données factices (5 minutes)
5. Tâche 6 : CharCNN URL (à relancer une fois)
6. Tâche 7 : DistilBERT sur le texte HTML
7. Tâche 8 : ResNet18 sur les captures d'écran
8. Tâche 9 : fusion des modalités
9. Tâche 10 : évaluation finale et comparaison complète
10. Tâche 11 : complément du benchmark RQ3
11. Tâche 12 : explicabilité et erreurs
12. Tout lancer d'un coup
13. Mes recommandations
14. Dépannage et check-list finale

1. Vue d'ensemble du Sprint 2

Le Sprint 1 a produit les entrées des modèles (`data/features/`) et les baselines MLlib. Le Sprint 2 entraîne un modèle deep par modalité, les **fusionne**, puis compare **tous** les modèles de façon rigoureuse pour répondre à RQ1 et RQ2.



Tâche	Script	Entrée	Sortie principale	Durée estimée (RTX 3050)
6	`url_charcnn.py`	url_feat	`data/emb/url_*.parquet`	2 à 3 min
7	`html_distilbert.py`	html_text	`data/emb/html_*.parquet`	30 à 50 min
8	`img_resnet18.py`	img_224	`data/emb/img_*.parquet`	20 à 35 min
9	`fusion.py`	data/emb + tech_feat	`results/fusion_ablation.csv`	5 à 15 min
10	`evaluate_final.py`	results/preds_*	`results/rapport_sprint2.md`	1 à 2 min
11	`benchmark_rq3.py` (fait, complément optionnel)	Parquet brut	`results/rq3_benchmark_grille.csv`	≈ 2 h (option)
12	`explain.py`	modèles + preds	`results/erreurs_*.csv`, figures	3 à 10 min

Les durées sont des estimations (pauses de refroidissement comprises), pas des mesures : le vrai temps s'affiche à la fin de chaque script et est noté dans `results/timings.csv`.

TROIS PROTOCOLES, LES MÊMES POUR TOUS LES MODÈLES

Protocole	Entraînement	Test	Sert à
rq1	trainval (7 879 pages)	test (2 724)	RQ1 : la fusion bat-elle chaque modalité seule ?
rq2	période « ancienne » (4 667)	période « récente » (3 385)	RQ2 : tient-on face aux nouvelles campagnes ?
rq2ctrl nouveau	mêmes pages que rq2, tirées au hasard, même taille	le reste	le « contrôle de taille » prévu : sépare l'effet du temps de l'effet du manque de données

Validation commune. Dans chaque protocole, les mêmes 10 % des pages d'entraînement (choisies à partir du `sample_id`) sont mises de côté pour **tous** les modèles deep. Elles servent à l'arrêt anticipé, au choix d'un seuil, et surtout à la fusion « stacking » : comme aucun encodeur ne les a vues, la fusion apprend sur des scores honnêtes. C'est pour cela que le CharCNN doit être relancé une fois (2 minutes).

2. Ajouter les fichiers au projet

Le dossier `phish360/sprint2/` de la bibliothèque du projet (ou l'archive `phish360_sprint2.zip`) contient les fichiers prêts. Copie-les à la racine de ton projet (`C:\Users\moham\Desktop\phish360-projet-main` ou ton dépôt Git) en gardant la même arborescence :

Fichier	État	Rôle
`src/deep_common.py`	nouveau	protocoles, validation commune, métriques, formats de sortie (utilisé par tous les scripts deep)
`src/url_charcnn.py`	**remplacé**	tâche 6, même modèle, branché sur `deep_common` + protocole rq2ctrl
`src/html_distilbert.py`	nouveau	tâche 7
`src/img_resnet18.py`	nouveau	tâche 8
`src/fusion.py`	nouveau	tâche 9
`src/evaluate_final.py`	nouveau	tâche 10
`src/benchmark_rq3.py`	**remplacé**	tâche 11, ajoute `--grid`, `--repeat`, `--out` (le comportement par défaut ne change pas)
`src/explain.py`	nouveau	tâche 12
`src/check_deep.py`	nouveau	vérifie l'environnement deep (GPU, paquets, données)
`src/make_fake_data.py`	nouveau	données factices pour tester sans le dataset
`requirements.txt`	**remplacé**	PySpark selon la version de Python, transformers, shap, tabulate
`setup_venv.ps1`	**remplacé**	PyTorch CUDA 12.6 au lieu de 12.1
`run_sprint2.ps1`	nouveau	enchaîne les tâches 6 à 12 avec journaux dans `logs/`

Puis, dans Git (optionnel mais conseillé avant de lancer les gros calculs) :

```
git checkout -b sprint2-deep
git add src requirements.txt setup_venv.ps1 run_sprint2.ps1
git commit -m "Sprint 2 : scripts deep (taches 6 a 12)"
```

Ne copie pas de dossier `data/`, `models/` ou `results/` par-dessus les tiens : ils contiennent tes résultats du Sprint 1 (baselines, CharCNN, RQ3). Les nouveaux scripts **ajoutent** des lignes à `results/deep_results.csv` sans effacer les autres.

3. Installations

Toutes les commandes se tapent dans le terminal PowerShell de VS Code, à la racine du projet, environnement activé :

```
.\.venv\Scripts\Activate.ps1
```

3.1 VÉRIFIER PYTORCH ET LE GPU

Ton PC a déjà PyTorch 2.11 avec CUDA (tâche 6 faite sur GPU). Vérifie :

```
python -c "import torch; print(torch.__version__, torch.cuda.is_available(),
torch.cuda.get_device_name(0))"
```

```
2.11.0+cu126 True NVIDIA GeForce RTX 3050 ...
```

Si tu lis `False` ou `+cpu`, réinstalle la version CUDA (pas besoin d'installer CUDA à part, seulement un pilote NVIDIA récent) :

```
pip uninstall -y torch torchvision
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu126
```

3.2 TORCHVISION (RESNET18)

Il doit venir du **même index** que torch, sinon pip remplace ton torch CUDA par un torch CPU :

```
python -c "import torchvision; print(torchvision.__version__)"
# s'il manque :
pip install torchvision --index-url https://download.pytorch.org/whl/cu126
```

3.3 LES AUTRES BIBLIOTHÈQUES

```
pip install -r requirements.txt
```

Ce qui est nouveau pour le Sprint 2 : `transformers`, `tokenizers`, `safetensors`, `huggingface_hub` (DistilBERT), `shap` (explicabilité), `xgboost` (modèle technique de la fusion), `tabulate` (tableaux du rapport automatique). Le fichier choisit tout seul PySpark 3.5.3 (Python 3.11) ou PySpark 4 (Python 3.13).

3.4 TÉLÉCHARGER LES MODÈLES PRÉ-ENTRAÎNÉS ET TOUT VÉRIFIER

```
python src\check_deep.py --download
```

```
[OK] Python: 3.13.x
[OK] PyTorch: 2.11.0+cu126
[OK] GPU: NVIDIA GeForce RTX 3050 ..., 6.0 Go, CUDA 12.6, calcul fp16 OK
[OK] torchvision: 0.26.0+cu126
[OK] transformers: ...
...
[OK] data/features/html_text: présent
[OK] DistilBERT: distilbert-base-multilingual-cased téléchargé (cache Hugging Face)
[OK] ResNet18: poids ImageNet téléchargés

Prêt pour le Sprint 2 !
```

Le téléchargement (≈ 540 Mo pour DistilBERT multilingue, 45 Mo pour ResNet18) n'a lieu qu'une fois ; les poids vont dans `%USERPROFILE%\.cache\huggingface` et `%USERPROFILE%\.cache\torch`.

Variable utile pour PySpark 4 (comme au Sprint 1, seulement pour les scripts Spark : baselines et RQ3) :
`$env:PYSPARK_SUBMIT_ARGS = "--conf spark.sql.ansi.enabled=false pyspark-shell"`
Les scripts deep (tâches 6 à 10 et 12) n'utilisent pas Spark : ils lisent les Parquet avec pandas.

4. Test à blanc avec des données factices

Avant d'occuper le GPU une heure, vérifie en 5 minutes que toute la chaîne tourne. Les données factices sont créées dans un dossier à part, **jamais** dans ton `data/` :

```
python src\make_fake_data.py demo_factice --pages 600
cd demo_factice
python ..\src\url_charcnn.py --epochs 2 --pause 0
python ..\src\html_distilbert.py --epochs 1 --batch 8 --pause 0 --max-len 64 --no-save-model
python ..\src\img_resnet18.py --epochs 1 --pause 0 --workers 0
python ..\src\fusion.py --seeds 1
python ..\src\evaluate_final.py --boot 200
python ..\src\explain.py --captures 4
cd ..
Remove-Item -Recurse -Force demo_factice
```

Chaque script doit finir sans erreur et afficher un tableau. Les scores n'ont **aucun sens** (données inventées) ; seul compte le fait que tout s'enchaîne.

5. Tâche 6 : CharCNN URL (à relancer une fois)

POURQUOI RELANCER

Le modèle ne change pas (embedding de caractères 32, convolutions 3/4/5 × 128 filtres, 40 époques max, patience 4 : les réglages de ta dernière version sont maintenant les valeurs par défaut). Ce qui change : la validation commune, le protocole rq2ctrl, et le fichier d'embeddings qui contient maintenant la probabilité et la partie (train / val / test) de chaque page, indispensables à la fusion.

COMMANDE

```
python src\url_charcnn.py
```

SORTIE ATTENDUE

```
Appareil : cuda (NVIDIA GeForce RTX 3050 ...)
=== rq1 : 7091 URL d'entraînement, 788 de validation, 2724 de test ===
  époque  1 : perte 0.5... PR-AUC validation 0.8...
  ...
  meilleure époque : 1x (PR-AUC validation 0.9..)
url_charcnn F1=0.8..  P=0.8..  R=0.7..  ROC-AUC=0.9..  PR-AUC=0.9..  | seuil val 0.xx ->
F1=0.8..
=== rq2 : ...      === rq2ctrl : ...
Comparaison avec la baseline URL TF-IDF (tâche 5) : (tableau)
```

Tu dois retrouver à ±0,01 près tes chiffres du 10/11 (F1 0,829 en RQ1, 0,778 en RQ2) : la validation n'est plus tirée de la même façon, de petites variations sont normales. Les nombres de pages exacts dépendent de ton index nettoyé.

6. Tâche 7 : DistilBERT sur le texte HTML

CE QUE FAIT LE MODÈLE

- Le texte = titre + « | » + texte visible (4 000 caractères, années masquées en `<annee>` depuis la tâche 4).
- Il est découpé en **tokens** (sous-mots). On garde les **256 premiers** (le début de la page : titre, menus, formulaire de connexion).
- DistilBERT (6 couches de Transformer, 66 M de paramètres pour la version anglaise, 135 M pour la multilingue) produit un vecteur par token ; le vecteur du token spécial `[CLS]` résume la page.
- Une petite tête (Dropout → 256 → ReLU → 1) donne le score « phishing ». Les 256 nombres servent d'**embedding** pour la fusion.
- **Fine-tuning** : tout le réseau est réentraîné, avec un pas très petit pour DistilBERT ($2 \cdot 10^{-5}$) et plus grand pour la tête (10^{-3}), échauffement puis décroissance linéaire, 4 époques max, arrêt anticipé (patience 2) sur la PR-AUC de validation.
- Précision mixte **fp16** : deux fois moins de mémoire et plus rapide sur la RTX 3050.

COMMANDES

```
# essai rapide (≈ 5 min) pour voir la vitesse et la mémoire
python src\html_distilbert.py --only rq1 --epochs 1

# la vraie exécution, 3 protocoles
python src\html_distilbert.py
```

Option	Par défaut	Quand la changer
`--model`	distilbert-base-multilingual-cased	`distilbert-base-uncased` pour comparer avec la version anglaise (plus petite, plus rapide)
`--max-len`	256	512 pour lire plus de texte (≈ 2,5× plus lent ; mettre `--batch 8 --accum 2`)
`--batch / --accum`	16 / 1	« CUDA out of memory » → `--batch 8 --accum 2`
`--freeze`	non	DistilBERT figé, seule la tête apprend : très rapide, possible sur CPU, mais moins bon (sert de point de comparaison)
`--no-save-model`	non	ne pas écrire les 3 × 540 Mo de `models/` (mais la tâche 12 `--tokens` en a besoin)

SORTIE ATTENDUE

```
10603 textes découpés en tokens en 2x s ; longueur médiane ..., xx % tronqués à 256
=== rq1 : 7091 pages d'entraînement, 788 de validation, 2724 de test ===
    lot    0/444  perte 0.69..
    lot  100/444  perte 0.3..
  époque 1 : perte 0.2..  PR-AUC validation 0.9..  (xxx s)
  ...
html_distilbert F1=0.9..  P=...  R=...  ROC-AUC=0.9..  PR-AUC=0.9..  | seuil val ...
Comparaison avec la baseline HTML TF-IDF (tâche 5) : (tableau)
```

À regarder : la PR-AUC de validation doit monter dès la 1re époque ; si la perte reste à 0,69 (= hasard), le pas est trop grand ou le texte est vide (voir dépannage). Objectif : battre HTML TF-IDF (F1 0,913 en RQ1, 0,884 en RQ2).

7. Tâche 8 : ResNet18 sur les captures

CE QUE FAIT LE MODÈLE

- Les captures 224 × 224 de la tâche 4 sont décodées à la volée et normalisées avec la moyenne / l'écart-type d'ImageNet (comme prévu au Sprint 1).
- **ResNet18 pré-entraîné** sur ImageNet (formes, logos, textes, mises en page) ; sa dernière couche est remplacée par une tête 512 → 256 → 1.
- Fine-tuning complet : pas 10^{-4} pour le corps, 10^{-3} pour la tête, décroissance en cosinus, 10 époques max, patience 3, fp16.
- **Augmentation** à l'entraînement : recadrage aléatoire léger (80-100 % de l'image) et petites variations de luminosité / contraste. Pas de miroir horizontal : il inverserait le texte et les logos.
- Pages sans image exploitable : exclues de l'entraînement ; au test elles reçoivent une image noire, et la fusion est prévenue par la colonne `img_ok`.

COMMANDES

```
python src\img_resnet18.py --only rq1 --epochs 1      # essai rapide
python src\img_resnet18.py                            # vraie exécution
```

Options : `--batch 16` si la mémoire manque, `--workers 0` si le chargement bloque sous Windows, `--freeze` pour le corps figé (comparaison « extraction de features » contre « fine-tuning »).

SORTIE ATTENDUE

```
10603 pages, 10 5xx images exploitables (x.x% manquantes)
=== rq1 : 70xx images d'entraînement, 7xx de validation, 2724 de test ===
    lot    0/22x  perte 0.6..
  époque  1 : perte 0.4..  PR-AUC validation 0.8..  (xx s)
  ...
img_resnet18 F1=0.8..  ...
```

À quoi s'attendre : l'image seule est en général la modalité la plus faible pour Phish360 (l'EDA a montré des captures de mêmes dimensions dans les deux classes, donc pas de raccourci) mais elle apporte une information **différente** (logos imités, formulaires), utile à la fusion.

8. Tâche 9 : fusion des modalités

LES TROIS STRATÉGIES

Modèle	Type	Principe	Point fort
fusion_stacking	tardive	une régression logistique combine les 4 probabilités (URL, HTML, image, XGBoost technique). Elle apprend **uniquement** sur la validation commune, que les encodeurs n'ont jamais vue.	pas de fuite, 5 paramètres, coefficients lisibles
fusion_concat	intermédiaire	chaque modalité → 64 nombres, concaténation, petit MLP	peut apprendre des interactions entre modalités
fusion_gated	intermédiaire + attention	une « porte » donne un poids à chaque modalité pour chaque page, puis somme pondérée	on voit sur quelle modalité le modèle s'appuie

Les deux MLP utilisent le **modality dropout** (une modalité entière masquée au hasard à l'entraînement, pour ne pas tout miser sur la meilleure) et la moyenne de **3 graines** (l'écart-type entre graines est noté dans `f1_graines_std`). La modalité technique est le vecteur des 35 features (log + standardisation) pour les MLP, et un XGBoost (`tech_xgb`, aussi évalué seul) pour le stacking.

> **Pourquoi deux familles ?** Les embeddings des pages d'entraînement viennent d'encodeurs qui ont appris sur ces mêmes pages : ils sont « trop beaux ». Un MLP entraîné dessus peut surestimer la modalité qui a le plus sur-appris. Le stacking évite ce piège (il apprend sur la validation). Comparer les deux est en soi un résultat à discuter dans le rapport.

ABLATION : RÉPONSE DIRECTE À RQ1

Stacking et concat sont refaits sur les **15 combinaisons** de modalités (URL seule, URL + HTML, …, les 4). On voit ainsi ce que chaque modalité apporte.

COMMANDES

```
python src\fusion.py                          # 3 protocoles + ablation
python src\fusion.py --only rq1 --no-ablation   # rapide
python src\fusion.py --modalites url html tech  # si la tâche 8 n'est pas faite
```

SORTIE ATTENDUE

```
=== rq1 : 7091 train, 788 validation, 2724 test ===
  modalités : url (128 dims), html (256 dims), img (256 dims), tech (35 dims)
  tech_xgb         F1=0.9..  ...
  fusion_stacking  F1=0.9..  ...
  fusion_concat    F1=0.9..  ...
  fusion_gated     F1=0.9..  ...
  ablation : 15 combinaisons x 2 stratégies
  (les 8 meilleures combinaisons, triées par PR-AUC)
```

Fichiers : `results/fusion_ablation.csv`, `results/fusion_poids.csv`, `results/fusion_portes_*.parquet`, `models/tech_xgb_*.json`, et les lignes ajoutées à `results/deep_results.csv`.

9. Tâche 10 : évaluation finale et comparaison complète

```
python src\evaluate_final.py
```

Le script relit les prédictions de **tous** les modèles (`results/preds_*.parquet` : baselines MLlib, deep, fusion) et recalcule tout de la même façon. Pour chaque protocole :

- **Tableau complet** : F1, précision, rappel, ROC-AUC, PR-AUC, F1 avec le seuil choisi sur la validation, temps d'entraînement ;
- **Intervalles de confiance à 95 %** du F1 et de la PR-AUC (bootstrap : 1 000 tirages des pages de test). Deux modèles dont les IC se chevauchent largement ne sont pas vraiment différents ;
- **Tests appariés** : meilleure fusion contre chaque autre modèle, sur les mêmes pages : différence de F1 avec son IC, et test de McNemar (les deux modèles se trompent-ils sur les mêmes pages ?) ;
- **Verdict RQ1** automatique : meilleure fusion contre meilleure modalité seule, « significativement mieux / moins bien / non significatif » ;
- **RQ2** : F1 du contrôle aléatoire contre F1 temporel, et la « perte due au temps » de chaque modèle.

FICHIERS PRODUITS (À METTRE DANS LE RAPPORT)

Fichier	Contenu
`results/rapport_sprint2.md`	tous les tableaux et verdicts, en Markdown (copiable dans le rapport LaTeX)
`results/final_comparaison.csv`	une ligne par protocole × modèle, avec les IC
`results/final_significativite.csv`	meilleure fusion contre chaque modèle
`results/final_rq2_derive.csv`	F1 rq1 / contrôle / rq2 et perte due au temps
`figures/final_f1_rq1.png` , `final_f1_rq2.png`	F1 de tous les modèles avec IC, couleur = famille
`figures/final_pr_rq1.png` , `final_pr_rq2.png`	courbes précision-rappel du meilleur modèle de chaque famille
`figures/final_rq2_derive.png`	contrôle contre temporel, par modèle
`figures/final_ablation.png`	F1 de chaque combinaison de modalités (RQ1)
`figures/final_cout.png`	temps d'entraînement de chaque modèle (lien avec RQ3)

GABARIT DU TABLEAU DE COMPARAISON (À COMPLÉTER AVEC TES CHIFFRES)

Famille	Modèle	F1 RQ1	F1 RQ2	PR-AUC RQ1	PR-AUC RQ2
Baseline MLlib	URL TF-IDF + LR	0,807	0,694	0,905	0,911
Baseline MLlib	HTML TF-IDF + LR	0,913	0,884	0,962	0,943
Baseline MLlib	Techniques GBT	0,934	0,867	0,984	0,941
Baseline MLlib	Concaténation + LR	0,934	0,846	0,983	0,976
Deep unimodal	URL CharCNN	0,829	0,778	0,927	0,887
Deep unimodal	HTML DistilBERT	…	…	…	…
Deep unimodal	Image ResNet18	…	…	…	…
Classique	Techniques XGBoost	…	…	…	…
Fusion deep	stacking / concat / gated	…	…	…	…

Chiffres connus : Sprint 1 et tâche 6 (10/11). Les « … » sont remplis par `evaluate_final.py`.

10. Tâche 11 : complément du benchmark RQ3

Le benchmark de base est fait (5 runs). `benchmark_rq3.py` accepte maintenant la grille complète et des répétitions, ce qui répond aux deux limites notées dans le rapport (grille partielle, un seul run). Les figures affichent alors la moyenne et l'écart-type.

```
$env:PYSPARK_SUBMIT_ARGS = "--conf spark.sql.ansi.enabled=false pyspark-shell"
python src\benchmark_rq3.py --grid --repeat 3 --out results\rq3_benchmark_grille.csv
```

9 configurations × 3 répétitions = 27 runs, environ 2 h avec les pauses : à lancer en fin de journée, PC branché. Version courte (sans grille, 3 répétitions, ≈ 45 min) : `python src\benchmark_rq3.py --repeat 3 --out results\rq3_benchmark_rep.csv`. L'option `--out` évite d'écraser `results/rq3_benchmark.csv` déjà présenté.

Le coût du deep (entraînement et pages/s en inférence) est noté automatiquement dans `results/timings.csv` et `results/deep_results.csv` (colonnes `train_s`, `inference_pages_s`) : c'est un bon complément à RQ3 (« Spark prétraite 54 pages/s sur 4 cœurs, DistilBERT en infère … pages/s sur le GPU »).

11. Tâche 12 : explicabilité et erreurs

```
python src\explain.py                          # SHAP + poids des modalités + erreurs (rq1 et
rq2)
python src\explain.py --gradcam 6 --tokens 5   # + Grad-CAM du ResNet18 et mots clés de
DistilBERT
```

Analyse	Sortie	Comment la lire
SHAP sur les features techniques	`figures/shap_tech_rq1.png`, `shap_tech_importance_rq1.csv`	un point = une page ; à droite = pousse vers phishing ; couleur = valeur de la feature (rouge = haute). Compare au top 10 GBT du Sprint 1 (text_len, n_links...).
Poids des modalités	`figures/fusion_poids.png`	coefficients du stacking et portes moyennes du modèle gated, par protocole : la modalité dominante change-t-elle en RQ2 ?
Analyse des erreurs	`erreurs_rq1.csv`, `erreurs_resume_rq1.txt`, `figures/erreurs_marques_rq1.png`	faux positifs / négatifs triés par confiance ; taux d'erreur selon le texte, l'image, la marque imitée ; quelle modalité seule avait raison quand la fusion s'est trompée.
Planche des captures	`figures/erreurs_captures_rq1.png`	les phishing manqués et les légitimes bloquées les plus « sûrs d'eux » : très parlant à l'oral.
Grad-CAM (option)	`figures/gradcam_rq1.png`	zones claires = ce qui a fait monter le score « phishing » (formulaire, logo ?).
Mots DistilBERT (option)	`tokens_distilbert_rq1.csv`	mots qui ont le plus poussé vers phishing (gradient × entrée) pour quelques pages.

12. Tout lancer d'un coup

Une fois le test à blanc réussi, tu peux tout enchaîner avec un seul script (journaux dans `logs\tacheN.log`) :

```
powershell -ExecutionPolicy Bypass -File .\run_sprint2.ps1
# reprendre à la tâche 8 après une coupure :
powershell -ExecutionPolicy Bypass -File .\run_sprint2.ps1 -From 8
```

Ordre imposé : 6, 7, 8 (dans n'importe quel ordre entre eux) → 9 (a besoin des trois) → 10 (a besoin de tout) → 12. Le complément RQ3 (11) est indépendant et se lance à part.

Ordre	Commande	Durée estimée
0	`python src\check_deep.py --download`	2-5 min
1	`python src\url_charcnn.py`	2-3 min
2	`python src\html_distilbert.py`	30-50 min
3	`python src\img_resnet18.py`	20-35 min
4	`python src\fusion.py`	5-15 min
5	`python src\evaluate_final.py`	1-2 min
6	`python src\explain.py --gradcam 6 --tokens 5`	3-10 min
(option)	`python src\benchmark_rq3.py --grid --repeat 3 --out results\rq3_benchmark_grille.csv`	≈ 2 h

13. Mes recommandations

POUR LA QUALITÉ DES RÉSULTATS

1. **Ne jamais régler quoi que ce soit sur le test.** Tous les choix (époque, seuil, poids de la fusion) se font sur la validation commune. Si tu essaies plusieurs réglages (max-len 256 contre 512, multilingue contre anglais), compare-les sur la PR-AUC de **validation** affichée pendant l'entraînement, puis ne rapporte le test que du réglage retenu.
2. **Présenter le F1 avec son IC 95 %** : avec ≈ 2 700 pages de test, l'IC du F1 fait environ ± 0,01. Une différence de 0,005 entre deux modèles n'est pas un résultat ; le script le dit pour toi.
3. **Pour RQ2, s'appuyer sur rq2ctrl.** Sans lui, on ne peut pas dire si la baisse vient du temps ou du plus petit entraînement (4 667 pages). La métrique à mettre en avant : la « perte due au temps ».
4. **RQ2 et le seuil** : le Sprint 1 a montré que les AUC restent hautes alors que le rappel chute. Présente aussi la colonne « F1 seuil val. » (seuil choisi sur la validation) et la PR-AUC, qui ne dépend pas du seuil.
5. **Garder le multilingue par défaut** pour DistilBERT (pages en tchèque, allemand, roumain, portugais, russe). Si le temps le permet, lancer aussi `--model distilbert-base-uncased` une fois sur rq1 : c'est une ligne de comparaison intéressante (et rapide).
6. **Fine-tuning contre figé** : lancer aussi `--freeze` pour DistilBERT et ResNet18 (rapide) donne deux lignes de plus qui montrent l'intérêt du fine-tuning. Elles apparaissent sous les noms `html_distilbert_gele` et `img_resnet18_gele`. Attention : relance ensuite la version complète en dernier, car la fusion lit le dernier embedding écrit.
7. **Discuter honnêtement la fusion** : si le stacking fait aussi bien que les MLP, c'est un résultat (la fusion tardive suffit, et elle est plus robuste et plus lisible). Si une fusion ne bat pas significativement les features techniques (F1 0,934), le dire : le test de McNemar et l'ablation servent précisément à ça.

POUR LE DÉROULÉ

1. Faire d'abord le test à blanc (section 4), puis un essai `--only rq1 --epochs 1` de DistilBERT pour mesurer le temps réel par époque.
2. Lancer DistilBERT et ResNet18 l'un après l'autre (pas en parallèle : 6 Go de mémoire GPU ne suffisent pas pour les deux).
3. Garder les pauses (`--pause`) si le PC chauffe ; surveiller la température avec `nvidia-smi -l 5` dans un 2e terminal.
4. Après chaque tâche, commit Git des petits résultats (`results/*.csv`, `results/figures/`) : ils sont autorisés par le `.gitignore`. Les modèles (`models/`) et embeddings (`data/emb/`) restent hors Git.
5. Copier `results/rapport_sprint2.md` et les figures `final_*.png`, `shap_tech_rq1.png`, `erreurs_captures_rq1.png` dans le rapport LaTeX du Sprint 2.

PLAN CONSEILLÉ POUR LE RAPPORT FINAL

Partie	Contenu	Sources
Modèles deep	architecture de chaque encodeur, réglages, courbes d'apprentissage (logs)	sections 5 à 7 de ce guide, `logs/`
RQ1	tableau complet, figure F1 + IC, ablation, verdict significatif ou non	`rapport_sprint2.md`, `final_f1_rq1.png`, `final_ablation.png`
RQ2	tableau rq2, perte due au temps, modalité la plus robuste	`final_rq2_derive.png`, `final_rq2_derive.csv`
RQ3	passage à l'échelle Spark (grille + répétitions) et coût du deep	`rq3_*.png`, `final_cout.png`
Explicabilité	SHAP, poids des modalités, erreurs typiques, Grad-CAM	figures de la tâche 12
Limites	une seule exécution des encodeurs, axe temporel approximatif (année de copyright), embeddings d'entraînement optimistes pour les MLP	

14. Dépannage et check-list finale

Symptôme	Cause probable	Solution
`CUDA out of memory`	lot trop gros	DistilBERT : `--batch 8 --accum 2` (ou `--max-len 192`) ; ResNet : `--batch 16`. Fermer les autres programmes qui utilisent le GPU.
`ValueError: Input contains NaN` ou « Le réseau produit des NaN »	précision fp16 instable (cartes GTX 16xx)	ResNet18 passe tout seul en fp32 sur GTX 16xx ; sinon ajouter `--no-amp` (ResNet18 et DistilBERT), et `--batch 16` pour ResNet sur 4 Go
DistilBERT trop lent (GTX 1650, 4 Go)	carte plus modeste que la RTX 3050	mesurer 100 lots avec `--only rq1 --epochs 1` ; si > 1 min 30 : `--batch 16 --max-len 128 --epochs 3`
`Appareil : cpu` alors qu'il y a un GPU	torch CPU installé	section 3.1 (réinstaller avec `--index-url .../cu126`)
le chargement des images reste bloqué	processus de chargement sous Windows	`--workers 0`
`OSError` / pas d'accès à huggingface.co	réseau, proxy, antivirus	relancer `check_deep.py --download` sur une autre connexion ; ou télécharger une fois puis travailler hors ligne avec `$env:HF_HUB_OFFLINE="1"`
perte bloquée à 0,69, PR-AUC ≈ part de phishing	le modèle n'apprend pas	vérifier que `html_text` contient du texte ; baisser `--lr` (1e-5) ; essayer `--freeze` pour vérifier la chaîne
`[!] data/emb/img_rq1.parquet absent` dans la fusion	tâche 8 pas faite pour ce protocole	lancer la tâche, ou `--modalites url html tech`
Spark : erreur ANSI sur une URL	PySpark 4	`$env:PYSPARK_SUBMIT_ARGS="--conf spark.sql.ansi.enabled=false pyspark-shell"`
évaluation : aucun fichier `preds_*.parquet`	lancé depuis le mauvais dossier	toujours lancer depuis la racine du projet

CE QUI A ÉTÉ TESTÉ AVANT LIVRAISON

Vérifications faites avant livraison (Linux, Python 3.13) : syntaxe de tous les scripts, génération des données factices, découpage des 3 protocoles, et évaluation finale complète (tableaux, IC, tests appariés, figures, rapport Markdown) sur ces données. Le test de bout en bout des scripts deep (tâches 6 à 9 et 12) sur CPU est indiqué dans le message qui accompagne ce guide. Ce qui ne peut pas être testé ici : ton GPU, Windows, le téléchargement des vrais poids pré-entraînés et les temps réels ; d'où le test à blanc (section 4) et l'essai rapide conseillés avant chaque gros calcul.

CHECK-LIST

☐	fichiers copiés dans le projet, commit sur une branche `sprint2-deep`
☐	`check_deep.py --download` : tout en [OK], GPU vu
☐	test à blanc avec `make_fake_data.py` réussi
☐	tâche 6 relancée (3 protocoles) ; F1 proche de 0,83 / 0,78
☐	tâche 7 DistilBERT (3 protocoles)
☐	tâche 8 ResNet18 (3 protocoles)
☐	tâche 9 fusion + ablation
☐	tâche 10 : `rapport_sprint2.md` et figures `final_*` relus
☐	tâche 12 : SHAP, erreurs, Grad-CAM
☐	(option) tâche 11 grille + répétitions
☐	commit des résultats (`results/*.csv` , `results/figures/`) et rédaction du rapport